

FlexRoads – Technology Stack Training

Instructions for the hands-on assignments.

1 Install and Set Up

1.1 Install DataFlex 27.0 Studio

1. Run the installer.
2. After installation is complete, use Windows Explorer to review the installed toolchains in
%ProgramData%\DataFlex\Toolchains.

1.2 Create a New Workspace

1. Name the workspace **Conference App**.
2. Do not place it inside the Users folder. Prefer a location such as C:\DataFlex Projects because of IIS permissions.

1.3 Create a Hello World App

1. Go to **Create New > Projects > Processes**.
2. *This creates a basic Windows project that you will later convert to a Tech Stack project.*
3. Name the project **HelloWorld**.
4. Enter the following code:

Use ui

```
Procedure ShowHello
    Integer iKey
    Showln "Hello world!"
    Inkey iKey
End_Procedure
```

Send ShowHello

1.4 Test run the application

1. Run the program using toolchain 27.0.0+windows-32.
This runs the project using the existing DataFlex runtime.
2. Change the toolchain to 0.1.5+native-64 and run the project again.
This runs the project using the native Tech Stack toolchain.

2 Part 2 – Create Your First WebAssembly App

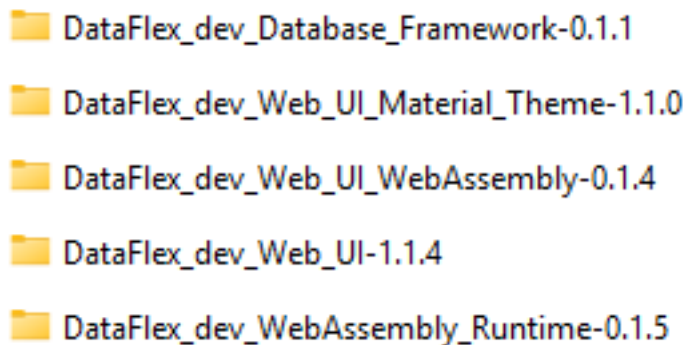
2.1 Extract the AppHtml Template

Extract **AppHtml Template.zip** into your **AppHtml** folder. The archive contains a basic **index.html**, icons, a manifest file, and an IIS configuration file.

2.2 Install Packages

Use the package manager to install the **Web UI WebAssembly** and **Web UI Material Theme** packages into your workspace.

Verify that the **DfPkg** folder has been created and contains these packages. Also confirm that **index.html** now includes the required script references.



2.3 Create the Project

Go to **Create New > Projects > Web UI WebAssembly** to create a new project. Name the project **ConferenceApp.src**, switch it to toolchain **0.1.5-wasm-32**, compile it, and confirm that **ConferenceApp.flx** is created in **AppHtml**.

2.4 Update index.html

Open **index.html** in your preferred text editor or in Studio. Find the line `const oWebApp = new df.WebClientApp("MyApp.flx");` and change it to `const oWebApp = new df.WebClientApp("ConferenceApp.flx");`.

2.5 Create the Dashboard

Because the drill-down model requires a default view, create the dashboard view as well. Go to **Create New > Views > WASM Dashboard** and use **oDashboard** as the object name.

2.6 Create an IIS Virtual Directory

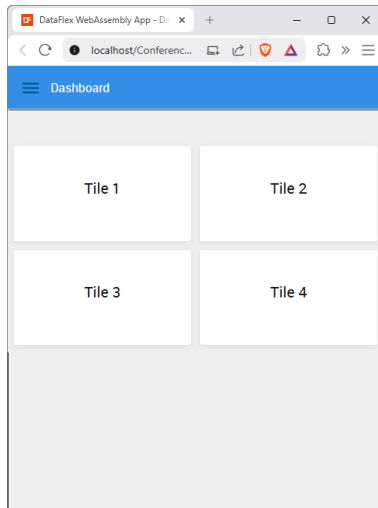
Open **Internet Information Services (IIS) Manager**. Expand the tree to **Default Web Site**, right-click it, and select **Add Application....** Use **Conference_App** as the alias and select your workspace's **AppHtml** folder as the physical path.

2.7 Update the Application URL

Open the project properties and change the application URL to `http://localhost/Conference_App/`.

2.8 Test the Application

Run the project from Studio. If everything is configured correctly, the newly created application should open in your browser.



3 Create the Presenter Entity and Views

3.1 Create the entity

In this step we'll create the Entity definition and Data Dictionary class.

1. Go to **Create New > Class Creation & Import > New Entity** to create a new entity package.
2. Name the package **PresenterDataDictionary.pkg**.
3. Update the code so the entity is named **Presenter**, the class is named **Presenter**, **phEntity** is set correctly, and the meta tag values are correct.
4. Rename the primary key field to **PresenterId**.
5. Add the following string fields: **FirstName**, **LastName**, **Company**, and **Bio**.
6. Create an index that includes **FirstName** and **LastName**.
7. Create a second index that includes **LastName** and **FirstName**.
8. Mark **FirstName** and **LastName** as required by using **Set Field_Option** within **Construct_Object**.
`Set Field_Option (RefTable(Self.FirstName)) DD_Required to True`
9. Set the entity's **Connection** meta tag to **conf_data**.

3.2 Create and Initialize the Database

Now we'll add code that creates the database when the app starts up for the first time.

1. Create a new empty dataflex source file named **CreateAndSyncDb.pkg**.
2. Add **Use** statements for **System\FileSystem.pkg** and **PresenterDataDictionary.pkg**.
3. Create an object based on the **cObject** class with the name **oCreateAndSyncDb**. Inside that object, create an instance of **cPresenterDataDictionary** named **oPresenterDD**.
4. Write a procedure named **InitializeDb** that checks whether the database file exists by using the **C_SqliteDb** constant defined in **ConferenceApp.src** together with the **File_Exists** command. If the file does not exist, call **CreateDatabase** on **oPresenterDD**.
5. Add a **Use** statement to **ConferenceApp.src** to include this package. Place it inside the **oWebApp** object and call **InitializeDb** during object creation.
6. Test the code by placing a breakpoint inside **InitializeDb** and starting the project in the debugger. Step through the code to confirm that **CreateDatabase** runs the first time and is skipped on subsequent runs.

3.3 Create the Presenter Zoom View

Now we'll create a zoom view for the presenter.

1. Use the **Create New > Views > WASM View Zoom** template to create a zoom view named **ZoomPresenter.wo**.
2. Add a **Use** statement for **PresenterDataDictionary.pkg**.
3. Inside **oZoomPresenter**, create an **oPresenterDD** object based on **cPresenterDataDictionary**, then set the **Main_DD** and **Server** properties to point to it.
4. Remove the existing groups and add three **cWebForm** controls and one **cWebEdit** control by using the designer.
5. Bind the forms to **oPresenterDD.FirstName**, **oPresenterDD.LastName**, and **oPresenterDD.Company**. Bind the **cWebEdit** control to **oPresenterDD.Bio**.
6. Set appropriate labels and, if desired, rename the objects for clarity.

3.4 Create the Presenter Select View

To get into our zoom view we also need a select view which we'll create in this step.

1. Use the **Create New > Views > WASM View Select** template to create a select view named **SelectPresenter.wo**.
2. Add a **Use** statement for **PresenterDataDictionary.pkg**.
3. Inside the view object, create an **oPresenterDD** object based on **cPresenterDataDictionary**, then set the **Main_DD** and **Server** properties to reference it.
4. Insert a **cWebColumn** after **oColumn1**.
5. Set the new column's **psCSSClass** to **RowCaption**, set the **piWidth** of both **oColumn1** and the new column to **50**, and set **piListColSpan** of **oColumn2** to **2**.
6. Bind the columns to **oPresenterDD.FirstName**, **oPresenterDD.LastName**, and **oPresenterDD.Company**. Optionally adjust the object names and captions.
7. Uncomment the **WebRegisterPath** line inside **oDetailButton** and change **oZoomView** to **oZoomPresenter**. Make the same change for **oNewButton**.
8. Optionally update dashboard tile 2 so it navigates to the presenter select view.

3.5 Test and Create a Record

Run the application and verify that all steps completed successfully. You should now be able to create your first presenter record.

4 Part 4 – Download presenters from Rest API

4.1 Configure the HTTP client

Edit **CreateAndSyncDb.pkg** to prepare it for implementing our synchronization logic by adding an http client object.

1. Use statements for **System\Http.pkg** and **System\Json.pkg**.
2. Inside the **oCreateAndSyncDb** object, create an instance of **cHttpClient** named **oHttpRequest**.
3. Set **psEndpoint** to the service base URL: `https://abs01.daelab.net/`.

4.2 Implement SynchronizeData

Now we'll create a procedure named **SynchronizeData** inside the **oCreateAndSyncDb** object that will download the presenters from the REST service and adds them to the database.

1. Define an array of **Presenter** structs named **aPresenters**.
2. Call **HttpGet** on **oHttpRequest** to retrieve presenter data from `"/ConferenceData/API/v1/Presenter"`.
3. Check whether the response status is **200**. If it is not, show an error message and return.
4. Use **HttpResponseToDataType** on **oHttpRequest** to deserialize the response directly into **aPresenters**.
5. Start a transaction. Within the transaction, first clear the table with **ZeroFile oPresenterDD**.
6. Loop through the presenters in **aPresenters** and create a record for each one using **oPresenterDD**. Note that you can use **UpdateAllFields** on **oPresenterDD** to assign field values from each **Presenter** struct.

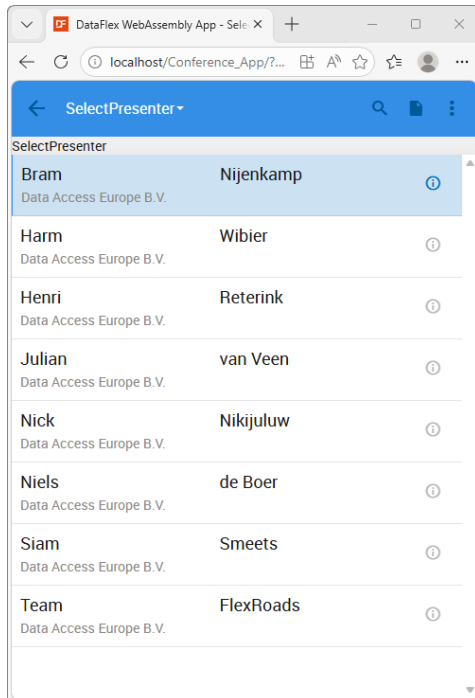
4.3 Call our synchronization logic from a dashboard tile

Now we'll change the welcome dashboard tile to call our SynchronizeDb procedure.

1. Inside **Dashboard.wo** rename **oWelcomeTile** to **oSyncTile**.
2. Change Tile 1 inside **psHtml** into Synchronize.
3. Set **pbServerOnClick** to true.
4. Implement **OnClick** to call **SynchronizeData** of **oCreateAndSyncDB**.

4.4 Test and run

Now it is time to run / debug the application and if successful we should end up with a filled presenter table.



The screenshot shows a web browser window with the title 'DataFlex WebAssembly App - SelectPresenter'. The address bar shows 'localhost/Conference_App/?...'. The page content is titled 'SelectPresenter' and displays a table of presenter information. The table has two columns for names and a third column for company details. The first row is highlighted in blue.

SelectPresenter		
Bram	Nijenkamp	Data Access Europe B.V.
Harm	Wibier	Data Access Europe B.V.
Henri	Reterink	Data Access Europe B.V.
Julian	van Veen	Data Access Europe B.V.
Nick	Nikijuluw	Data Access Europe B.V.
Niels	de Boer	Data Access Europe B.V.
Siam	Smeets	Data Access Europe B.V.
Team	FlexRoads	Data Access Europe B.V.

5 Add search

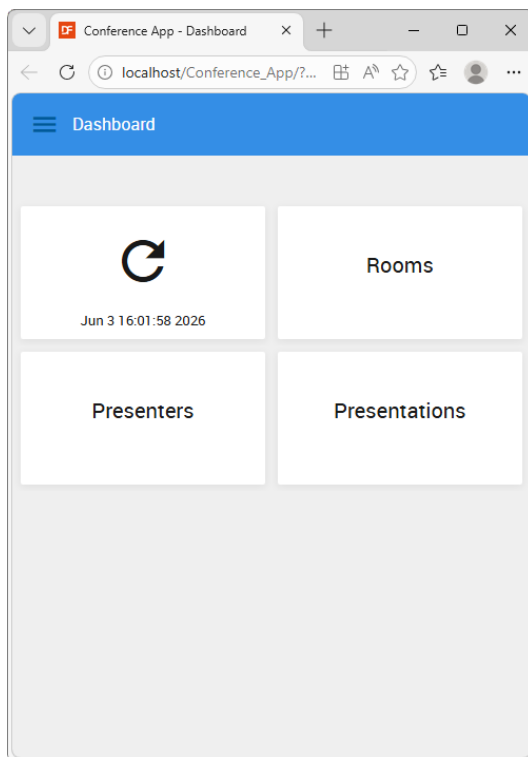
5.1 Add presentations and rooms

Before we can add the search we'll skip forward a few steps. Add / overwrite the following files with the ones from the **Step 5-1.zip** or alternatively switch to branch Step-5-1 in the git repository.

AppHtml/CssStyle/application.css
AppSrc/ConferenceApp.src
AppSrc/CreateAndSyncDB.pkg
AppSrc/Dashboard.wo
AppSrc/PresentationDataDictionary.pkg
AppSrc/RoomDataDictionary.pkg
AppSrc/SelectPresentation.wo
AppSrc/SelectPresenter.wo
AppSrc/SelectRoom.wo
AppSrc/SettingsDataDictionary.pkg
AppSrc/ZoomPresentation.wo
AppSrc/ZoomPresenter.wo
AppSrc/ZoomRoom.wo

5.2 Test run

Test run the application. The dashboard should look like this:



5.3 Implement Presentation Search

In this step we'll extend the **SelectPresentation** view with the ability to filter presentations on their title by adding a constrain to the Presentation DD.

1. Open **SelectPresentation.wo** and find the **oPresentationDD** object
2. Add a string property called **psFilter**
3. Add a procedure **OnConstrain**
4. In this procedure check if **psFilter** is set, if so add a contains constrain on the Title field

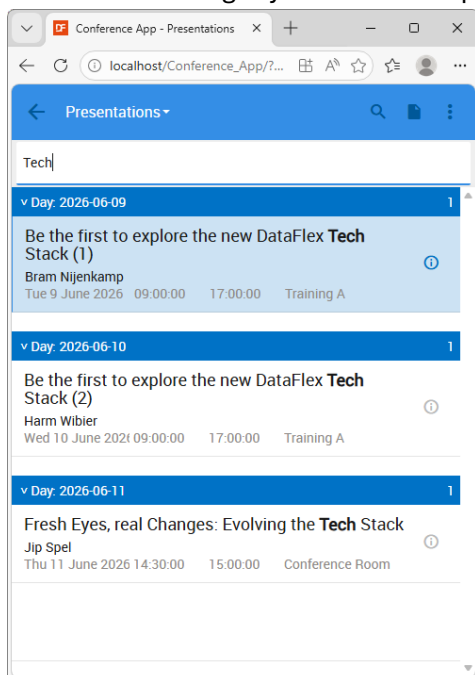
5.4 Add a Search Form

In this step, you will add a search form to the **SelectPresentation** view so users can filter the list as they type.

1. Add a **cWebForm** above the list.
2. Set the following properties on the form:
 1. **pbShowLabel = False**
 2. **psPlaceholder = "Type to search..."**
 3. **pbServerOnInput = True**
3. Implement the **OnInput** event with these steps:
 1. Use **WebGet psValue** to retrieve the current input.
 2. Set **psFilter** of **oPresentationDD**.
 3. Rebuild the constraints.
 4. Refresh the list by using **FindDDRRecordInBuffer**.
 5. **WebSet psHighlight** of **oPresentationDD_Title**.

5.5 Test your changes

Now test the changes you made in the previous steps.



6 Wrap the App for Android

6.1 Create the ZIP File

To package the application as an Android APK, first create a ZIP file containing the contents of the **AppHtml** folder. This package should include the **.flx** file, the WebAssembly runtime, JavaScript files, CSS files, and any required subfolders.

1. Zip all files in the **AppHtml** folder, including all subfolders.

6.2 Upload the Package to the Build Server

Next, upload the ZIP file to the Android Build Service so it can generate the APK.

1. Open **<https://abs01.daelab.net/>** in your browser.
2. Complete the required fields and upload the ZIP file.
3. Choose the option to generate a keystore.
4. Click **Start Build**.
5. Wait for the build to finish.

6.3 Test the App on Your Phone

After the build completes, install the APK on your phone and verify that the app works correctly.

1. Scan the QR code with your phone's camera app.
2. Download the **.apk** file.
3. Install the APK. You may see warnings because it is not recognized as a store app.
4. After installation, launch the app from your home screen.
5. After the first run, test the app again in airplane mode.

6.4 Test the App in an Android Emulator

If you have an Android emulator available, you can test the APK there instead of using a physical device.

1. Download the **.apk** file.
2. Start the emulator.
3. Install the APK. In Android Studio, you can usually install it by dragging the file onto the emulator window.